MySQL Query Optimization: Approach, Implementation, and Results

1. MySQL Query Optimization with Multi-Column Indexes

(Background)

- The query took 2 seconds to execute. It aggregated data from a high volume table (over 1 million rows) by joining two related tables in a Many-to-One relationship, and included GROUP BY and the aggregate functions MAX and MIN

Type	Text		Duration
SQL / User	SELECT DISTINCT sub.body_part	AS `body_part`,¶	2s
SQL / User	SELECT DISTINCT sub.body_part	AS `body_part`,¶	2s
SQL / User	SELECT DISTINCT sub.body_part	AS `body_part`,¶	2s
SQL / User	SELECT DISTINCT sub.body_part	AS `body_part`,¶	2s
SQL / User	SELECT DISTINCT sub.body_part	AS `body_part`,¶	2s
SQL / User	SELECT DISTINCT sub.body_part	AS `body_part`,¶	2s
SQL / User	SELECT DISTINCT sub.body_part	AS `body_part`,¶	2s

- Based on the SQL logs output from TypeORM-generated queries, I examined the execution plan (EXPLAIN) and detailed execution plan (EXPLAIN ANALYZE)
- The EXPLAIN output showed ALL and index in the type column across different subqueries. This indicated that indexing was not properly applied, resulting in full table scans or full index scans
- Furthermore, EXPLAIN ANAYZE revealed that the deleted_at IS NULL condition was applied after “Table SCAN”, which contributed to poor JOIN performance
- The SQL query was optimized to reduce execution time, but TypeORM's abstraction made it difficult to express the same logic
    - Raw SQL was avoided due to maintainability issues and the risk of SQL Injection

    - Using raw SQL was excluded due to difficulty in maintenance and vulnerability to SQL injection
- Single-column indexes are flexible for optimizing individual queries, but performance can be degrade in complex condition searches that require scanning multiple indexes

(Reason for adoption)

- A multi-column index can improve SQL query performance by reducing unnecessary data scans, but it is effective only when the indexed columns are used in the defined order
- A multi-column index can improve SQL query search performance by promoting Index Scan in the SQL execution plan and reducing the likelihood of Full Table Scan
- By applying a multi-column index on (deleted_at, id), queries can first filter delete_at IS NULL and then efficiently scan the index using id, avoiding a full table scan
- For example, a multi-column index on (deleted_at, exercise_id, weight) was applied to optimize query performance. The query first filters deleted_at IS NULL, selects exercises using exercise_id, and retrieves the maximum weight. This help improve query performance by minimizing unnecessary data scans

(Achievements/ Reflection)

- Applying multi-column indexes reduced the query duration from 2 seconds to 0.27 seconds, improving performance by approximately 7 times.

Type	Text	Duration
SQL / User	SELECT `MaxWeightPerExercise`.`created_at` AS `MaxWeight	0.040s
SQL / User	SELECT `MaxWeightPerExercise`.`created_at` AS `MaxWeight	0.032s
SQL / User	SELECT `MaxWeightPerExercise`.`created_at` AS `MaxWeight	0.023s
SQL / User	SELECT `MaxWeightPerExercise`.`created_at` AS `MaxWeight	0.024s
SQL / User	SELECT `MaxWeightPerExercise`.`created_at` AS `MaxWeight	0.029s
SQL / User	SELECT `MaxWeightPerExercise`.`created_at` AS `MaxWeight	0.019s
SQL / User	SELECT `MaxWeightPerExercise`.`created_at` AS `MaxWeight	0.011s

- In the execution plan, we observed the type value changed from ALL to ref, indicating that the leading columns of the multi-column index were properly utilized in the query condition

- In the detailed execution plan, we observed Table SCAN was replaced with Index Lookup, indicating that only the necessary rows were scanned by utilizing the appropriate index in the query. This significantly reduced unnecessary data access and improved overall query efficiency
- Learned that the type value in the execution plan is an important indicator of query performance and how SQL accesses tables
- Learned how to analyze execution plans in depth to identify inefficiencies such as unnecessary Table Scans, and how to design indexes to resolve them effectively
- Learned that the order of columns in a multi-column index affects how data is sorted and accessed, impacting query performance
- Learned that generally the column that is filtered first should be placed first in the index. However, there is no single correct answer, so **it is important to verify the execution plan and measure query performance during optimization**

(**Areas for improvement**)

- Although the current optimization is effective with around 1 million records and only 25 types of exercises, the effectiveness of the multi-column index may decrease as the number of exercise types grows

2. Enhanced Aggregation Performance with Multi-Column Indexes and Denormalization

(Background)

- The query performance degradation was primarily due to the high cost of 'GROUP BY' operations during data aggregation
- Although the queried data was rarely updated, aggregation had to be executed on every request
- To address this structural inefficiency, denormalization was considered by introducing a separate table to store pre-aggregated data

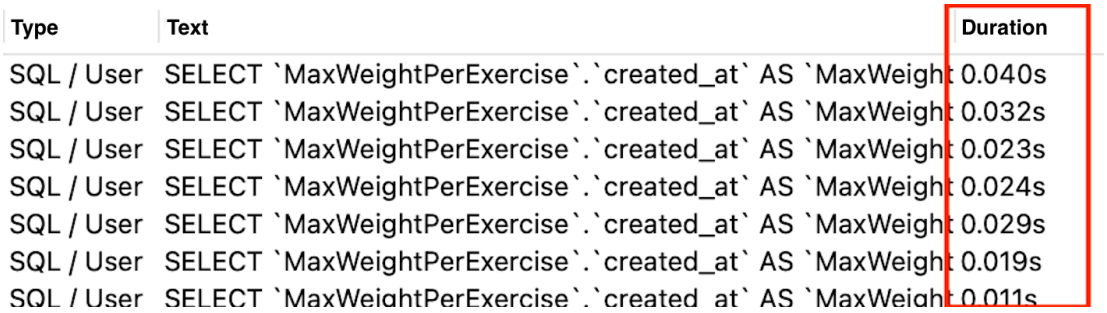
(Reason for adoption)

- By applying denormalization, frequently calculated data can be pre-stored, enabling faster responses after the initial request

- Since the queried data is rarely updated in this project, read performance can be maximized without significant concerns about data consistency
- However, if the aggregated data is not updated in real time, data integrity issues may arise. To address this, a synchronization mechanism was implemented and applied

(Achievements/ Reflection)

- Applying multi-column indexes and denormalization reduced the query duration from 2 seconds to 0.025 seconds, improving performance by approximately 80 times



Type	Text	Duration
SQL / User	SELECT `MaxWeightPerExercise`.`created_at` AS `MaxWeight	0.040s
SQL / User	SELECT `MaxWeightPerExercise`.`created_at` AS `MaxWeight	0.032s
SQL / User	SELECT `MaxWeightPerExercise`.`created_at` AS `MaxWeight	0.023s
SQL / User	SELECT `MaxWeightPerExercise`.`created_at` AS `MaxWeight	0.024s
SQL / User	SELECT `MaxWeightPerExercise`.`created_at` AS `MaxWeight	0.029s
SQL / User	SELECT `MaxWeightPerExercise`.`created_at` AS `MaxWeight	0.019s
SQL / User	SELECT `MaxWeightPerExercise`.`created_at` AS `MaxWeight	0.011s

- By additionally applying denormalization, the query execution time was approximately 10 times faster compared to when only multi-column indexes were used, which took around 27 seconds
- A separate table was created to store aggregated data to maintain data integrity
  - When source data was inserted or updated, the corresponding aggregated data was synchronized
  - When source data was deleted, the aggregated data was recalculated to reflect the latest state
- Learned that when handling frequently updated data, it is important to minimize synchronization overhead by updating only the necessary fields or designing periodic update mechanisms
- Learned that denormalization is a powerful optimization technique for environments where read performance is critical, but it must be applied carefully, considering the risks of data inconsistency and potential degradation of write performance

- Learned that denormalization should be considered as a last resort when query performance remains insufficient even after SQL tuning and indexing optimization

3. Fast Query Response with Multi-Column Indexes and Redis Caching

(Background)

- The table for aggregation data was added for SQL optimization when using denormalization with multi-column indexes
- As part of optimizing query performance, an aggregation table was introduced using a combination of multi-column indexing and denormalization
- However, the table structure would need to be modified whenever new aggregation fields are added or existing ones are removed, which reduces structural flexibility and increases maintenance overhead
- To improve structural flexibility and reduce maintenance overhead, an alternative optimization method was considered. As a result, Redis, **an in-memory NoSQL database optimized for high-speed data processing**, was selected

(Reason for adoption)

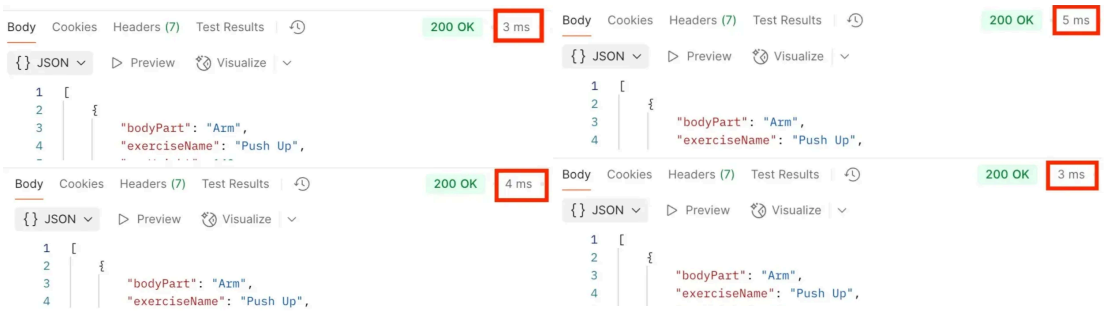
- NoSQL database that stores data in a key-value format, allowing much faster access than disk-based RDBMS
- Since the target data in this project is infrequently updated, using Redis does not raise concerns about performance degradation due to frequent write operations
- Additionally, even if further aggregation fields need to be added later, the system can be scaled flexibly without requiring schema changes
- By applying the Cache Aside strategy, the system first checks Redis on each request, and if the data is not found, it fetches it from the database and stores it in the cache
- This strategy allows repeated requests to be served instantly from the cache, improving response time and reducing unnecessary database

access

- Furthermore, Redis is well suited for future features that require real-time analytics, such as displaying the most frequently performed exercises or real-time user rankings
- For these reasons, the system was designed to separate responsibilities: the RDBMS stores the source data, while Redis caches responses to time-consuming queries in order to maximize performance

(Achievements/ Reflection)

- Applying multi-column indexes and Redis reduced the query duration from 2 seconds to 0.004 seconds, improving performance by approximately 500 times



- The query execution time was approximately 6 times faster compared to when multi-column indexes and denormalization were used, which took around 27 seconds
- A issue of data inconsistency between Redis and RDBMS is addressed by synchronizing the corresponding aggregated data when the new data is added
- A potential issue of data inconsistency between Redis and RDBMS was resolved by comparing and updating the data stored in Redis whenever exercise records are added
- Learned that while Redis provides excellent read performance, data integrity must be carefully considered
- Learned that SQL tuning and indexing should be prioritized for performance optimization, but caching systems like Redis are more than just supplementary tools and they can play a key role in read-heavy systems such as news feeds, search, and ranking features